Middlesex University London

# CST3144 Full Stack Development

# COURSEWORK BRIEF

**Deadline: 14th December at 11pm.**

**Type of CW:** Individual Work.

**Demonstration:** the demonstration will be recorded by the Lecturer for internal and external moderation purposes.

**IMPORTANT. A submission could receive zero marks if it fails any of the following requirements:**

• The student must **book an available slot for the demo** of a CW**, by the CW submission deadline,** via the related "Individual Demonstration Booking System" on the Module Page, **AND** the work must be **demonstrated within the demonstration weeks scheduled,** during the **demo slot booked** (**IMPORTANT**: see more details and guidance on demo booking at **section 2.2.4**, and for guidance on other important related aspects at **section 2.2**).

• The work must be **demonstrated** and **explained satisfactorily during demonstration**, i.e., student can explain what the code does.

**Use of AI tools**

**Software Code: AMBER – Assistive scope**
GenAI tools may be used to help you write sections of your code more efficiently. However they are not a substitute for understanding and being able to edit your code. Note in particular the following:

- GenAI should only be used to create short sections of your code. Using GenAI to develop your whole project in one go is expressly prohibited. You are responsible for making sure all the parts of your code work together correctly.
- If you use GenAI tools to generate the code for you without understanding the technologies or without being able to describe your code properly during demonstration, then you will get a low mark for this Module.
- Any use of GenAI must be declared as part of your submission (see the table “Record and Acknowledgement of GenAI Use” in the module handbook.

**Demonstration: RED (no scope)**
The use of GenAI during the demonstration is prohibited.

**Task.** For this coursework, you need to create the Front-End of a fictitious web app, which allows students and their parents to buy after school classes and activities. Then, you need to build the Back-End for the app. MongoDB will be used for storing the data, and data exchange between the app and the database will be done through REST API implemented using Express.js. The Back-End will run on a Node.js server.

## 1. Submission Instructions

Your CW submission will consist of **a single zip file**.

The zip file must be **no more than 10MB in size**, and it must contain the following elements:

- your code in **2 folders** respectively related to:
  - your **Vue.js App.**
  - your **Express.js App** (**do not include the 'node_modules' folder**, otherwise the zip will be too big to submit).
- A README file with the following links:
  - **[Vue.js App]** the link to your GitHub Repository.
  - **[Vue.js App]** the link to your GitHub Pages from where the app can directly run.
  - **[Express.js App]** the link to your GitHub Repository.
  - **[Express.js App]** the link to the render.com route that returns all the lessons.
- All **collections** (including the **'lesson'** and **'order'** collections) exported from your MongoDB Atlas. See here for how to export collection in MongoDB Compass : https://docs.mongodb.com/compass/current/import-export#export-data-froma-collection
- the requests you created in Postman. See here for how to export requests in Postman: https://learning.postman.com/docs/getting-started/importing-and-exporting/exporting-data

**Instructions for the text area when uploading your submission:**

- please include the same information as requested for the README file.

**Overall Requirements and Instructions**

• You can use an external CSS library such as Bootstrap. Make sure the library file or online link is included in the submission.

**A submission could receive zero marks if it fails any of the following requirements:**

- **[GitHub Repositories]** t
  - The code of the Vue.js App must be hosted in a GitHub repository, with **at least 10 commits**, and the code of the Express.js App must be hosted in another GitHub repository, with **at least 10 commits**.
- **[Front-End]**
  - the App must be implemented by using Vue.js framework. Other frameworks or related technologies (e.g., React, AngularJS, Svelte, Apache, XAMPP) are NOT allowed**.**
  - Any JavaScript library is NOT allowed if it duplicates or replaces features provided by Vue.js framework. Check with the tutor if not sure.
- **[Back-End and Connection details for the Front-End]**
  - The Back-End server must use “Node.js”; others such as Apache or Xampp are not allowed.
  - The REST API must be developed with “Express.js”.
  - The Front-End data access must be achieved with “promise” using “fetch” function; “XMLHttpRequest” or library such as axios:js are not allowed.
  - The connection to MongoDB (in your Express.js App) must use the native Node.js driver only; libraries like Mongoose are not allowed.
- **[Hosting]**
  - The Front-End Vue.js app must be hosted and demonstrated on **GitHub Page**s.
  - The Back-End server must be hosted on **render.com**. Other hosting services are not allowed. The address of your hosted server must include your student number, e.g. https://M001234567-after-school.onrender.com
  - The data must be stored in **MongoDB Atlas** and retrieved via your Express.js App; local MongoDB or any other databases are not allowed.

## Marking Criteria

### Core Functionality (60%)

It is expected that most students will be able to complete all these tasks.

- **[Front-End] "Display List of Lessons" functionality (7%):**
  - A. there should be at least 10 lessons, and each lesson should have 5 spaces (or availability) (1%).
  - B. each lesson should have at least (5%): Subject (1%), Location (1%), Price (1%), Spaces (or availability: this indicates how many spaces are left) (1%), a Font Awesome icon (or an Image) (1%).
  - C. v-for must be used for the display of the lesson list (1%).
- **[Front-End] Sort functionality (8%):**
  - A. the user can choose to sort the lessons by one of the following attributes: subject, location, price, or spaces (i.e. availability) (5%).
  - B. there must be an option to sort in ascending or descending order (order dependent on the sorting attribute selected), which should work for each of the attributes (3%).
- **[Front-End] "Add to Cart" functionality (5%):**
  - A. each lesson must have an "Add to Cart" button (1%).
  - B. the button is always visible, and only enabled when space is larger than 0 (1%).
  - C. clicking the button once (related interactions implemented by using v-on) will add one space to the shopping cart, reducing the remaining space by one (2%).
  - D. once there is no more space, i.e. space = 0, the "Add to cart" button should be disabled but still visible, i.e. clicking it will not further reduce "space" nor add lessons to the cart (1%).
- **[Front-End] "Shopping Cart" functionality (5%):**
  - A. the shopping cart button should only be enabled after at least one lesson is added to cart (1%).
  - B. clicking the shopping cart button should show the cart page, and clicking the button again goes back to the lesson page (1%).
  - C. the shopping cart, in the cart page, should show all the lessons added (1%).
  - D. in the shopping cart page, the user should be able to remove lessons from the shopping cart; the removed lesson is added back to the lesson list (in the lesson page) (2%).

- **[Front-End] Checkout functionality (6%):**
  - A. the checkout is part of the shopping cart page (not part of the lessons page) (1%).
  - B. the "checkout" button is always visible and only enabled (clickable) after valid "Name" and "Phone" are provided (2%).
  - C. the "Name" must be letters only and the "Phone" must be numbers only; the check must be done using JavaScript (suggestion: regular expressions) (2%).
  - D. clicking the "checkout" button should display a message confirming the order has been submitted (1%).
- **[Back-End] MongoDB should have (8%):**
  - A. a collection for lesson information (minimal fields: topic, price, location, and space) (4%).
  - B. a collection for order information (minimal fields: name, phone number, lesson IDs, and number of space) (4%). Suggestion: the element lessonIDs can contain 1 or more lesson IDs, depending on how many different kinds of lessons you have in your order. Other solutions could be accepted: in fact, how you design this is not the primary aspect of this module, therefore it is up to you to find a reasonable solution that works satisfactorily.
- **[Back-End] Middleware Functions implemented in the Express.js Server should include (8%):**
  - A. a "logger" middleware that outputs all requests to the server console; the student needs to be able to inspect and explain the log (4%).
  - B. a static file middleware that returns lesson images, or an error message if the image file does not exist; the student needs to be able to test and demonstrate these aspects (4%).
- **[Back-End] REST API implemented in the Express.js Server should include and the following (you will also be asked to test and explain the routes during the demonstration) (7%):**
  - A. one GET route /lessons, which returns all the lessons as a Json (2%). Example:

    *[*
    *{' topic': 'math', 'location': 'Hendon', 'price': 100, 'space': 5},*
    *{' topic': 'math', 'location': 'Colindale', 'price': 80, 'space': 2},*
    *{' topic': 'math', 'location': 'Brent Cross', 'price': 90, 'space': 6},*
    *{' topic': 'math', 'location': 'Golders Green', 'price': 95, 'space': 7},*
    *]*

  - B. one POST route, which saves a new order to the “order” collection (2%).
  - C. one PUT route, which can update any attribute in a lesson of the “lesson” collection (suggestion/requirement: this will be used, after an order is submitted, for updating the available spaces: updating to any number the available spaces, and not just increasing or decreasing) (3% if the requirement is fully covered, 1% if the solution is just increase or decrease).
- **[Front-End] Fetch Functions implemented in the Front-End should include (6%):**
  - A. one fetch that retrieves all the lessons with GET (2%).
  - B. one fetch that saves a new order with POST after it is submitted (2%).
  - C. one fetch that updates the available lesson space with PUT after an order is submitted (2%).

## Challenge tasks (20%)

These tasks are not covered in detail in the lectures and labs, so you will have to do some research and thinking to complete them. It is not expected that everyone will be able to complete these tasks.

You **must** be prepared to explain your solution and how it works, describing how you implemented your solution in the front-end, middleware, server, and/or database (as applicable). A solution for this part of the coursework could receive **zero** if you are not able to explain it.

These tasks should be implemented using the techniques taught in the module, without using any additional libraries.

- **[Challenge] Search Functionality (6%)**
  - The user can search for a lesson, without specifying which attribute to search on. For example, searching for “a” should return all the lessons with “a” in its “title” or “location” or “price” or “availability”.
  - The search should be implemented in the back-end and the front-end should receive the results via REST API.
  - This task is worth at most 3% if the search is implemented only in the front-end (i.e. by filtering the results returned from the /lessons route).

- **[Challenge] Teachers (8%)**
    - The database also stores information about the teachers for each lesson. Each lesson has a first name, a last name, and is associated with one or more lessons that they teach. The user can choose a teacher and view all the lessons taught by that teacher.

- **[Challenge] Enrolment (6%)**
    - Implement a GET route /enrolment/:lessonid which takes as input the id of a lesson, and returns the enrolment number for that lesson – i.e. the number of slots that have already been purchased.

## Demonstration (20%)

During your demonstration session, you will show your completed web app and the code for your project and explain how it works.

Remember that the demonstration is a **mandatory** part of the assessment – students who do not attend the demonstration will receive zero for the whole submission.

In order to ensure that the demonstration runs smoothly, please make sure you have the following open on your device before your allotted start time:

- The front-end of your app on Github Pages
- The logs for your server on render.com
- Your database on Mongodb Atlas
- The source code for your front-end and back-end in Visual Studio Code (or another IDE)
- Your API requests in Postman

During the demonstration, you will be asked to do the following:

- **Demonstrate Front-End functionality (5%):**
    - Show the front-end of your website behaving as intended i.e. displaying lessons, adding lessons to cart, placing orders, etc.
- **Inspect server log (3%):**

- Show the console log output in render.com from your 'logger' middleware and explain what the logs mean. (1% if the student is able to show the log output but not able to explain it)

- **Demonstrate requests in Postman (5%)**
  - For each of the following REST API routes, you should have a Postman request prepared in advance and be ready to demonstrate it:
  - A. the GET route /lessons, which returns all the lessons as a Json (1%).
  - B. the POST route, which saves a new order to the "order" collection (2%).
  - C. the PUT route, which can update any attribute in a lesson of the "lesson" collection (2%).

- **Answer questions about your code (7%)**
  - During the demonstration we will ask 5-10 short questions about different parts of your code and their function. Make sure you understand your code so that you are able to answer these questions correctly.

- **Explain your solutions to "challenge" tasks:**
  - Explain your solutions to any challenge tasks you complete, showing in the code how you implemented your solution in the front-end and back-end. (See above for the marks associated with each challenge task)

## 2.2 Extension, Late Penalty, Incentive System and Demonstration Process

### 2.2.1 Extension

All extensions must be applied through the *Extenuating Circumstances service*

(see Section "Extenuating circumstances" in the handbook for more details). Please do not contact the module leader for extension.

### 2.2.2 Late Penalty

The late penalty rule of the University is included in the rules of the extenuating circumstances. For more details read the Section "Extenuating circumstances:", which indicates you how to reach the information on late penalty rule available in MyMDX.

Please do not contact the module leader for late penalty.

### 2.2.3 An Incentive System for Motivating Students to Have their Demo in The First Weeks, and for Properly Balancing Demonstrations

Demonstrations of the assessment are scheduled in different weeks sessions. In order to distribute our Demonstrations, and to reward Students for completing on time:

1. [First Week] if a student books and gives the Demo in the first scheduled week, the student is rewarded with 4 points.

2. [Second Week] if a student books and gives the Demo in the second scheduled week, the student is rewarded with 2 points.

3. [Third Week] for the 3rd week there are 0 reward points.

4. [Exceeding Reward Points] if the sum composed of “grade achieved + plus Rewards Points” exceeds the maximum possible grade, any excess points are lost. For example, if a student has 4 rewards points and scores 98 on the assessment, the total grade becomes 100, using only 2 Reward Points, and the other 2 Reward Points exceeding are forfeited.

2.2.4 Demonstration Process and Important Requirements/Instructions

The following applies to the demonstration process of all coursework parts. A submission could receive zero marks if it fails any of the following requirements:

• The student must book an available slot for the demo of a CW, by the CW submission deadline, via the related "Individual Demonstration Booking System" on the Module Page.

• The work must be demonstrated within the demonstration weeks scheduled, during the demo slot booked.

Demonstration Process: Important Instructions (obtaining a grade, submission, demonstration and booking).

Requirements for obtaining a grade:

• to submit your work by the CW deadline AND

• to book your Demo by the CW deadline AND

• to attend and give your booked Demo.

Avoiding Issues related to Booking:

• when the booking system is created, there are enough slots for the number of students of the Module (everything is scheduled and planned according to the number of students, time, and available rooms).

• if you leave your booking at the last minute, there could not be the slot you prefer, or slots could have been all booked.

• accordingly, it is your responsibility and part of a valid submission to book your demo. Thus, if you try to book at the last minute and there are no more available slots, it is your fault and unfortunately you get 0 for that part; however, if you get an official extension/deferral from UniHelp, and the Lecturer is notified from UniHelp, your Demo can be scheduled at the next possible opportunity (in this case, after getting an official extension/deferral from UniHelp, contact the Lecturer for asking a new scheduling of your demo).

Avoiding Issues related to not attending your Booked Demo:

• it is your responsibility and part of a valid submission to attend your booked demo and give your demo satisfactorily; if you cannot attend your demo, and you have not important reasons, it is your fault and unfortunately you get 0 for that part.

• However, if you cannot attend the demo (you have booked) due to important reasons:

1. check if there are still available slots, and, in that case, feel free to book another one autonomously.

2. if there are no other available slots, and if you cannot really attend your booked demo due to important reasons, if you get an official extension/deferral from UniHelp, and the Lecturer is notified from UniHelp, your Demo can be scheduled at the next possible opportunity (in this case, after getting an official extension/deferral from UniHelp, contact the Lecturer for asking a new scheduling for your demo).